Python: Getting Started

The absolute first steps: getting Python running, the interactive prompt, your first script, and the handful of fundamentals needed to read and write simple programs. For real depth on any topic here, see the **Basics** sheet.

Get Python

```
python3 --version       # check whether Python is already installed
```

macOS and most Linux systems already have it as `python3` (you always run Python 3). To install or manage Python versions, see the **uv** cheat sheet.

The interactive prompt (REPL)

The REPL (Read-Eval-Print Loop) runs code one line at a time and prints each result immediately — ideal for trying things out.

```
python3                 # start the REPL (quit with exit() or Ctrl-D)
```

```
>>> 2 + 3
5
>>> name = "Alice"
>>> f"hi {name}"
'hi Alice'
>>> exit()
```

Running a script

Put code in a file whose name ends in `.py`, then run it from the terminal.

```
# hello.py
print("Hello, world!")
```

```
python3 hello.py        # -> Hello, world!
```

Indentation & code blocks

Python groups code by **indentation**, not braces. A block opens after a colon `:` and its lines are indented (4 spaces by convention). Indentation must be consistent — it is part of the syntax, not just style.

```
if 3 > 2:
    print("three is bigger")     # indented -> inside the if
    print("still inside")
print("outside the if")          # back to the left -> always runs

# Wrong indentation is an error, not just ugly:
# if True:
```

```
# print("x")        # IndentationError: expected an indented block
```

Comments

```
# A comment starts with # and is ignored by Python
x = 5        # comments can also sit at the end of a line
```

Output and input

```
print("Hello")            # Hello
print("a", "b", "c")      # a b c   (space between items, newline at the end)

# input() reads one line from the user and ALWAYS returns a string.
# (Commented here so it doesn't wait for input.)
# name = input("Your name: ")
# print("Hi", name)

# Convert ("cast") between types -- needed because input() gives a string
int("5")        # 5      string -> int
str(5)          # '5'    int -> string
float("3.14")   # 3.14   string -> float
# int(input("Age: "))    # read text, then convert it to a number
```

Reading an error (traceback)

When something goes wrong Python prints a *traceback*. Read it **bottom-up**: the last line names the error type and message; just above it is the file and line number where it happened.

```
Traceback (most recent call last):
  File "hello.py", line 2, in <module>
    print(age + 1)
          ^^^
NameError: name 'age' is not defined
```

Here `age` was used before being given a value. Errors you will meet early: `NameError` (unknown name), `TypeError` (wrong type of value), `IndexError` (position out of range), `SyntaxError` (a typo in the code itself).

Getting help (in the REPL)

```
type(42)        # <class 'int'>     what kind of value is this?
type("hi")      # <class 'str'>
dir(str)        # list every method a string has (as strings)
# help(str)     # print full documentation (press q to leave the pager)
```

A first taste

Just enough to read a simple program. Each of these has a fuller treatment on the **Basics** sheet.

```
# Variables: a name just points at a value (no type declarations)
```

```
age = 30
name = "Alice"
is_member = True

# Basic types: int, float, str, bool
type(3), type(3.0), type("x"), type(True)   # int, float, str, bool

# A collection: the list
fruits = ["apple", "banana"]
fruits.append("cherry")      # ['apple', 'banana', 'cherry']
fruits[0]                    # 'apple'

# Selection (choose what to do)
if age >= 18:
    status = "adult"
else:
    status = "minor"

# Iteration (do something for each item)
for fruit in fruits:
    print(fruit)

# A function (reusable, named block that can return a value)
def greet(who):
    return f"Hello, {who}!"

greet("Alice")               # 'Hello, Alice!'

# A class (a blueprint for objects that bundle data + behaviour)
class Dog:
    def __init__(self, name):
        self.name = name
    def bark(self):
        return f"{self.name} says woof!"

Dog("Rex").bark()            # 'Rex says woof!'
```